Pyspark most important interview questions

1. What is the key difference b/w Hadoop MapReduce and Spark in terms of performance and scalability?

Feature	Hadoop MapReduce	Apache Spark
Processing Model	Disk I/O heavy batch jobs	In-memory DAG, efficient transformations
Speed & Performance	Slower, especially for iterative jobs	Fast—thanks to caching and optimized execution
APIs & Development	Java-centric, verbose	High-level APIs across Scala, Python, Java, R
Workload Support	Batch-only	Batch, streaming, SQL, graph, ML unified engine
Fault Tolerance	Re-run failed tasks from disk	Lineage tracking and recomputation
Resource & Ecosystem	Hadoop-dependent	Flexible deployment, extensive integration
Ideal Scenarios	Simple ETL and batch workloads	Real-time, iterative, multi-paradigm workloads

Follow up: Spark writes data to disk. Why and what's the difference w.r.t MapReduce?

Spark writes data to disk only when we instruct by using storage levels in persist() or when data doesn't fit into memory and spilling into disk is required which makes it scenario specific.
Even though Spark and MapReduce when writing data into disk look the same, the difference can be seen in performance. Spark performs much better than MapReduce due to catalyst optimizer and dynamic optimization techniques like AQE.

2. What is the role of SparkContext in PySpark?

It's the **entry point** to Spark's execution engine. It connects your application to the Spark cluster, manages resources, distributes tasks, and coordinates execution.

In modern Spark, it's created automatically inside a `SparkSession` (`spark.sparkContext`). Without it, no Spark jobs can run.

anshika-gupta-81b8a9b9/

3. Explain Spark Architecture.

Components of Spark Architecture

- **Driver Program**
  - The main process that runs the Spark application.
  - Converts user code (in Python, Scala, Java, etc.) into a **DAG (Directed Acyclic Graph)** of stages.
  - Schedules tasks for execution across the cluster.
  - Contains the **SparkContext** which is the gateway to the Spark cluster.

- **Cluster Manager**
  - Responsible for allocating resources to Spark applications.
  - Can be **Standalone**, **YARN**, or **Mesos** (Kubernetes in modern setups).

- **Executors**
  - Worker processes running on cluster nodes.
  - Execute tasks assigned by the driver.
  - Store data in memory or on disk for caching.

Execution Flow

1. **User Code → RDD/Dataset/DataFrame Transformations**
   - Code gets translated into a **logical plan**.
2. **Logical Plan → Physical Plan**
   - Optimized by the **Catalyst optimizer**.
3. **Stages and Tasks**
   - Spark breaks the job into **stages** (based on shuffle boundaries).
   - Each stage has multiple **tasks** (same operation on different partitions).
4. **Cluster Manager Allocation**
   - Driver requests executors from the cluster manager.
5. **Execution on Executors**
   - Executors run tasks in parallel and send results back to the driver.

Key Points

- Spark uses **lazy evaluation** — transformations aren't executed until an action is triggered.
- Executors are **long-lived processes** (unlike Hadoop MapReduce which starts fresh JVMs per task).
- Spark stores data **in memory** for speed, reducing disk I/O compared to MapReduce.
- The driver monitors execution and re-runs failed tasks if needed.

anshika-gupta-81b939169/

4. What is the difference between RDDs, Dataframe and Dataset?

5. The driver monitors execution and re-runs failed tasks if needed.

Feature	RDD (Resilient Distributed Dataset)	DataFrame	Dataset
Abstraction Level	Low-level	High-level	High-level
Type Safety	Type-safe (in Scala/Java)	Not type-safe	Type-safe (only in Scala, not in PySpark)
Ease of Use	More complex APIs	Easier, SQL-like API	Similar to DataFrame
Performance	Less optimized (no Catalyst/Tungsten)	Optimized using Catalyst & Tungsten	Optimized (like DataFrame)
Serialization	Java serialization	Tungsten binary format	Encoders (efficient)
Use Case	Complex, fine-grained transformations	SQL operations, analytics	When compile-time type safety is required (Scala only)
Support in PySpark	✅ Yes	✅ Yes	❌ Not supported (only in Scala/Java)

6. What is query optimization in Pyspark?

Spark uses **lazy evaluation** — transformations aren't executed until an action is triggered.

Catalyst Optimizer Steps

1. **Analysis** – Checks syntax, resolves column names, validates schema.
2. **Logical Plan** – Represents the user's query logically (no execution yet).
3. **Optimization** – Applies rules like:
   - Predicate pushdown (filtering early)
   - Constant folding (evaluating constant expressions before execution)
   - Projection pruning (selecting only required columns)
4. **Physical Plan** – Decides execution strategy (e.g., broadcast join vs shuffle join).
5. **Code Generation** – Uses whole-stage codegen for faster execution.

Common Optimization Techniques in PySpark

- **Use DataFrame API instead of RDDs** – DataFrames get Catalyst/Tungsten benefits.
- **Avoid UDFs unless necessary** – UDFs are slower; use Spark SQL functions.
- **Column Pruning** – Select only the columns you need.

anshika-gupta-81b9391b9/

- **Predicate Pushdown** – Filter data early so less data flows through stages.
- **Broadcast Join** – For small tables, use `broadcast()` to avoid shuffling.
- **Cache/Persist** – For reused DataFrames, cache results in memory.
- **Partitioning** – Repartition or coalesce wisely to balance parallelism.
- **File Format** – Use columnar formats like **Parquet/ORC** for faster reads.

7. What is SparkSession in Pyspark?

Introduced in **Spark 2.0** to replace older separate contexts.It's a unified object that combines

- **SparkContext** (for RDDs)
- **SQLContext** (for SQL queries)
- **HiveContext** (for Hive support)

Every PySpark program starts by creating a `SparkSession`. It tells Spark:

- How to run (local or cluster)
- App name
- Configurations (memory, cores, etc.)

Without it, you can't access DataFrame APIs.

Creating a SparkSession:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("RetailDataAnalysis") \
    .master("local[*]") \
    .getOrCreate()
```

Parameters:

- `appName()` → Name for the Spark job (visible in UI).
- `master()` → Execution mode:
  - `"local[*]"` → Run locally using all cores.
  - `"yarn"`, `"mesos"`, `"spark://..."` → For cluster modes.
- `.getOrCreate()` → Creates a new session or returns an existing one.

anshi ota-81b939169/

8. Difference b/w wide & narrow transformation in Pyspark?

1. Narrow Transformations

- **Definition**: Transformations where **each input partition** contributes to **only one output partition**.
- **No shuffling** of data across the network.
- **Faster** because computation stays local to the partition.
- **Examples**:
  - `map()`
  - `filter()`
  - `union()`
  - `coalesce()` (without increasing partitions)
- **When to use**: If you can process data within the same partition without needing data from others.

2. Wide Transformations

- **Definition**: Transformations where **input partitions** contribute to **multiple output partitions**.
- **Involves shuffling** data across nodes in the cluster (network + disk I/O).
- **Slower** because shuffle is expensive.
- **Examples**:
  - `groupByKey()`
  - `reduceByKey()`
  - `join()`
  - `distinct()`
  - `repartition()`
- **Why slower**: Data is redistributed to new partitions → shuffle → sort → aggregation.

3. Shuffle Process in Wide Transformations

- Steps:
  - **Map stage** – Data is prepared for shuffle.
  - **Shuffle stage** – Data is moved across the cluster.
  - **Reduce stage** – Data is aggregated/processed after shuffle.

- **Optimization Tip**: Reduce shuffles as much as possible by using:
  - `reduceByKey()` instead of `groupByKey()`
  - `mapPartitions()` for custom processing.

anshika-gupta-81b939169/

4. Execution Plan

- **Narrow transformations** → executed in a **single stage**.
- **Wide transformations** → cause a **stage boundary** (new stage in DAG).
- Spark's **DAG scheduler** splits jobs into stages based on these transformations.

5. Key Interview Points

Feature	Narrow	Wide
Data movement	No shuffle	Shuffle
Speed	Faster	Slower
Stage creation	Same stage	New stage
Examples	`map`, `filter`	`join`, `groupByKey`

9. What is the use of coalesce() and repartition() in Pyspark?

Feature	Repartition	Coalesce
Purpose	Increases or decreases number of partitions.	Decreases number of partitions only.
Shuffling	Full shuffle of data across all nodes.	Minimizes shuffle, avoids full shuffle.
Use Case	When increasing partitions or evenly redistributing data.	When reducing the number of partitions for optimized output.
Performance Cost	Expensive due to full shuffle.	Inexpensive due to limited movement.
Typical Scenario	Before wide transformations like `join`, `groupBy`.	Before writing data to disk.
Data Redistribution	Even redistribution across partitions.	Narrows existing partitions (merging adjacent ones).

anshika-gupta-81b399169/

10. When to use cache() and persist() in Pyspark and what is the difference b/w the 2?

We use them when:

- The **same DataFrame/RDD** is used **multiple times** in a Spark application.
- We want to **avoid recomputation** each time it's used.
- The dataset is expensive to compute (e.g., involves **joins, aggregations, filtering large data**).
- We want to **improve performance** in iterative algorithms (e.g., ML model training, graph processing).

Example scenario:
If you perform an expensive transformation like `df.filter(...).groupBy(...).agg(...)` and plan to use the result in multiple actions (`count()`, `show()`, `write()`), you should cache/persist it.

Feature	`cache()`	`persist()`
Default Storage Level	Stores data in **MEMORY_ONLY**.	Allows specifying a **storage level** (default: MEMORY_ONLY).
Flexibility	Fixed storage level.	Flexible – can store in MEMORY_AND_DISK, DISK_ONLY, MEMORY_ONLY_SER, etc.
When to Use	When the dataset fits in memory and is reused multiple times.	When memory is limited or you need custom storage behavior.
Example	`df.cache()`	`df.persist(StorageLevel.MEMORY_AND_DISK)`

```
from pyspark import StorageLevel
# Using cache (MEMORY_ONLY)
df_cached = df.cache()

# Using persist with custom storage level
df_persisted = df.persist(StorageLevel.MEMORY_AND_DISK)

# Trigger actions
df_cached.count()
df_cached.show()
```

nika-gupta-81b939169/

💡 **Key Points for Interviews**

- `cache()` is just a shorthand for `persist(StorageLevel.MEMORY_ONLY)`.
- Use `persist()` if data might not fit in memory or you want to store serialized or on disk.
- Both are lazily evaluated — data is stored only after the **first action**.
- Always `unpersist()` when the cached/persisted dataset is no longer needed to free up resources.

11. What is the importance of partitions in Pyspark?

1️⃣ Parallelism & Performance

- Spark processes data in **parallel** by splitting it into **partitions i.e. MPP(Massive Parallel Processing)**
- Each partition is processed **independently** by a task on a Spark executor.
- More partitions → more tasks → better parallelism (up to a point).
- Too few partitions → underutilized CPU cores.
- Too many partitions → overhead in task scheduling.

2️⃣ Data Distribution

- Partitions determine **how data is distributed** across the cluster.
- Well-distributed partitions → balanced workload.
- Skewed partitions (uneven data) → **data skew** → performance bottlenecks.

3️⃣ Memory Management

- If a partition is too large → may cause **OutOfMemoryError** on executors.
- Proper partitioning ensures each partition fits into executor memory.

4️⃣ Shuffle Optimization

- During **wide transformations** (`groupBy`, `join`), Spark **shuffles** data b/w partitions.
- The number and size of partitions after shuffle impacts performance.
- Setting the right `spark.sql.shuffle.partitions` is critical for big datasets.

5️⃣ ETL Efficiency

- In data engineering pipelines:
  - **Before joins/aggregations** → increase partitions for parallelism.
  - **Before writing to storage (like ADLS, S3)** → reduce partitions to avoid too many small files.

anshika-gupta-81b939169/

```
# Check partitions
df.rdd.getNumPartitions()

# Repartition for parallelism
df = df.repartition(8)

# Coalesce to reduce partitions before writing
df = df.coalesce(2)
```

12. What are broadcast variables and why are they used?

- **Read-only shared variables** that are **cached on every executor node** in a Spark cluster.
- They allow you to **avoid sending the same data repeatedly** with each task.
- Instead, Spark broadcasts the data once to each node, significantly reducing **network I/O and serialization overhead**.

Why Use Them?

- Ideal for **small lookup tables, configuration settings, or reference data** used across many tasks.
- Especially useful when performing operations that need frequent access to the same static dataset.

Suppose you're processing a massive dataset of user activity and need to enrich it with country names based on country codes:

```
states = {"NY": "New York", "CA": "California", "FL": "Florida"}

broadcast_states = spark.sparkContext.broadcast(states)
df = spark.read.parquet("...")

def lookup_state(code):
    return broadcast_states.value.get(code, "Unknown")
df_rdd = df.rdd.map(lambda row: (row.user_id, lookup_state(row.state_code),
row.activity))
```

Use broadcast variables when:

- You have a **small-read-only dataset** that needs to be reused across many tasks or stages.
- You want to **reduce shuffle and serialization overhead**.

Avoid them when:

- The dataset is too large to fit in executor memory—could lead to OOM errors.
- The broadcasted data isn't used by enough tasks to justify the overhead.

13. What is the difference b/w df.show() and df.collect()?

Feature	`df.show()`	`df.collect()`
Purpose	Displays data in a **tabular format** in the console for quick preview.	Retrieves **all rows** from the DataFrame to the **driver** as a Python list of Row objects.
Default Behavior	By default shows **top 20 rows**. Can specify number of rows with `df.show(n)`.	Always returns **all rows** in the DataFrame unless filtered before calling.
Output Location	Prints to **console only** (doesn't return data).	Returns a **list** that you can store in a variable and process further.
Memory Impact	Lightweight; doesn't load the full dataset into driver memory.	Can be dangerous for large datasets – may cause **OutOfMemoryError** on the driver.
Use Case	For **quick inspection** or debugging.	When you **need to programmatically work** with all the data in the driver.

For large datasets, avoid `collect()` unless you are absolutely sure the data fits in memory — instead, use `take(n)` or `limit(n).collect()`.

14. What is lazy evaluation in Pyspark?

Lazy Evaluation in PySpark means that Spark does **not** execute transformations immediately when you call them.
Instead, it **builds a logical execution plan (DAG)** and only runs the computation when an **action** (like `show()`, `collect()`, `count()`, etc.) is triggered.

anshika-gupta-01bb3169/

How it works:

1. **Transformations** (`select()`, `filter()`, `map()`, etc.) → Only recorded in a plan; no actual execution.
2. **Action** (`show()`, `count()`, `collect()`) → Triggers Spark to optimize the plan and execute it on the cluster.

Advantages:

- **Optimization**: Spark can combine multiple transformations into a single stage (pipelining).
- **Efficiency**: Avoids unnecessary computation.

15. What are advantages of Delta Lake over traditional file formats?

Feature	Traditional File Formats (CSV, Parquet, etc.)	Delta Lake
ACID Transactions	❌ Not supported (risk of partial writes/inconsistent data)	✅ Guaranteed consistency with ACID transactions
Schema Enforcement	❌ Data with wrong schema may get written silently	✅ Rejects writes with schema mismatch
Schema Evolution	⚠️ Limited (Parquet allows but without validation)	✅ Add/modify columns with proper versioning
Data Versioning (Time Travel)	❌ Not available	✅ Access older versions of data for rollback, audit, or debugging
Upserts & Deletes	❌ Difficult (requires full file rewrite)	✅ Native support via `MERGE INTO`, `DELETE`, `UPDATE`
Performance (Indexing & Caching)	⚠️ Reads entire files, no transaction log optimization	✅ Optimized reads/writes with transaction log (`_delta_log`)
Streaming + Batch Unification	❌ Usually separate pipelines	✅ Same Delta table can be used for batch & streaming
File Compaction (Optimize)	❌ Manual and complex	✅ Built-in commands to compact small files for better performance

anshika-gupta-81b939169/

Delta Lake turns your data lake into a **transactional, reliable, and high-performance lakehouse**, solving common issues like inconsistent data, slow updates, and schema drift.

16. What happens when a Pyspark job runs OOM?

When a **PySpark job runs Out of Memory (OOM)**, it means the executors or driver have exhausted their allocated memory.

Stage	What Happens
1. Task Execution	Executors start processing data partitions in memory.
2. Memory Exhaustion	If the data, shuffle operations, caching, or joins exceed the executor's memory limit, Spark tries to spill data to disk (temporary files) to free memory.
3. Spill to Disk	Spark writes intermediate data to disk to avoid OOM. This slows down the job but prevents immediate failure—if spill space is enough.
4. GC Pressure	JVM Garbage Collector runs frequently to reclaim memory. If it spends too much time in GC (> ~98% of time), Spark throws a GC overhead error.
5. OOM Failure	If neither GC nor spilling can free enough space, you get an **OutOfMemoryError** (e.g., `Java heap space` or `GC overhead limit exceeded`). The stage fails.
6. Job Retry / Abort	Spark retries the failed stage (default 4 times). If it fails every time, the job aborts.

Common Causes

- Very large partitions (too much data for one executor).
- Wide transformations (e.g., large shuffles from `groupBy`, `join`).
- Caching huge datasets without enough memory.
- Skewed data causing one executor to handle much more data than others.

Prevention / Fixes

- **Increase memory**: `--executor-memory` and `--driver-memory`.
- **Increase partitions**: `repartition()` to reduce data per partition.

anshika-gupta-8-1a8a3169/

- **Use broadcast joins** for small datasets.
- **Persist wisely**: Use `persist(StorageLevel.DISK_ONLY)` if RAM is limited.
- **Enable spilling configs**: Tune `spark.shuffle.spill` and `spark.memory.fraction`.
- **Handle skew**: Use salting or skew join optimization.

In PySpark, if a job runs Out of Memory (OOM), Spark first tries to spill intermediate data to disk to free up RAM. If spilling and garbage collection can't help, the executor throws an `OutOfMemoryError` and the stage fails. Spark retries the stage (default 4 times), but if all retries fail, the job aborts.

Common causes are large partitions, skewed data, or caching huge datasets.
To fix it: increase executor/driver memory, repartition to reduce data per task, use broadcast joins for small datasets, and persist to disk instead of memory when RAM is limited.

17. What is AQE in Pyspark and why is it useful?

AQE is a Spark feature (enabled by `spark.sql.adaptive.enabled=true`) that **dynamically optimizes query plans at runtime** based on the actual data being processed, rather than relying only on static plans created during compile time.

Why it's useful:

1. **Handles Data Skew** – Detects uneven partition sizes and splits/rebalances them to avoid slow tasks.
2. **Optimizes Join Strategies** – Can switch from shuffle join to broadcast join if a dataset turns out smaller than expected.
3. **Reduces Shuffle Partitions** – Automatically merges small shuffle partitions to avoid excessive tasks and overhead.

Example:
In a retail analytics pipeline, if after filtering, the orders dataset becomes small, AQE can switch to a broadcast join automatically—saving shuffle time and boosting performance.

18. How would you handle skewed data in Pyspark?

When data is skewed, some partitions hold far more records than others, causing certain tasks to run much longer. This leads to slow stages and possible OOM errors.

Technique	How It Works	Example
Salting	Add a random "salt" key to the join column to spread skewed keys across multiple partitions, then remove salt after processing.	For a heavy `customer_id=123`, create `customer_id_salted = concat(customer_id,`

ansika-gupta-81b939169/

		`rand_int)` before join.
Broadcast Join	If one dataset is small, broadcast it to all executors to avoid shuffles.	`broadcast(df_small)` in `join`.
AQE Skew Join Optimization	Enable `spark.sql.adaptive.enabled` to let Spark split skewed partitions dynamically.	`spark.conf.set("spark.sql.adaptive.enabled", "true")`.
Filter Early	Reduce data size before joins or aggregations.	Apply `where()` before `join`.
Repartition by Key	Increase parallelism for the skewed key.	`df.repartition(200, "key")`.

💡 **Retail Example:**
If 70% of your transactions come from one store ID, salting that store ID before joins will ensure the workload is spread evenly across executors.

19. What is broadcast join and when should you use it?

A **broadcast join** sends a small DataFrame to all worker nodes so they can join it locally with their partition of the big DataFrame, avoiding a costly shuffle.

When to use:

- One DataFrame is **small enough** to fit in each executor's memory (rule of thumb: < 10 MB, but can be adjusted via `spark.sql.autoBroadcastJoinThreshold`).
- To **speed up joins** by avoiding network shuffle of large datasets.
- Especially useful in **star-schema** or **lookup table** joins (fact table + small dimension table).

```
df_result = df_large.join(broadcast(df_small),on="store_id",how="left")
df_result.show()
```

20. What is SPILL is Spark and why does it happen?

In **Spark**, **spill** happens when data being processed does not fit entirely into memory, so Spark temporarily writes (or *spills*) it to disk to continue processing.

akshika-gupta-81b99169/

Why does the spill happen?

- Spark operations like **shuffle**, **sort**, **groupBy**, **join**, or **aggregation** store intermediate data in memory.
- If there's **insufficient executor memory** (based on `spark.executor.memory`, `spark.memory.fraction`), Spark can't hold all intermediate data in RAM.
- To avoid **OutOfMemory (OOM)** errors, Spark *spills* this extra data to disk.

Common scenarios which cause spill?

1. **Large shuffles** → e.g., `groupByKey`, large joins, repartitions.
2. **Wide transformations** with huge intermediate datasets.
3. Data skew → one partition has too much data to fit in memory.
4. Too many tasks competing for limited executor memory.

Impact of spilling

- **Performance degradation** — disk I/O is much slower than memory.
- However, it allows Spark jobs to **complete successfully** instead of failing with OOM.

How to reduce spilling

- Increase memory per executor (`spark.executor.memory`).
- Increase shuffle memory fraction (`spark.memory.fraction`).
- Use **map-side aggregation** (`reduceByKey` instead of `groupByKey`).
- Avoid data skew (salting, repartitioning).
- Broadcast small tables instead of shuffling large ones.

21. What are Delta Lake's time travel features and how do they work?

Delta Lake Time Travel allows you to **query, restore, or roll back** data to a previous version of a Delta table.
It's essentially like having *built-in version control* for your data.

How it works

- Every write operation in Delta Lake creates a **new version** of the table (stored in the `_delta_log` transaction log).
- The transaction log stores **metadata + commit history** for each version.
- You can query **by version number** or **by timestamp**.

arshika-gupta-81b939169/

1️⃣ **Query by version number**

```
df = spark.read.format("delta")\
               .option("versionAsOf", 5).load("/path/to/table")
```

2️⃣ **Query by timestamp**

```
df = spark.read.format("delta")\
.option("timestampAsOf", "2025-08-01 12:00:00").load("/path/to/table")
```

Key points

- Time travel works as long as old data files are retained (controlled by `delta.logRetentionDuration` & `delta.deletedFileRetentionDuration`).
- Default retention is 30 days.
- Does not duplicate data — it uses copy-on-write storage to maintain versions.

anshika-gupta-81b99916